Pipeline Construction, Model Training, and Baseline Evaluation

Week 5, Day 2

Lumumba W. Victor
Trainer — Data Analyst — ML Expert

MediaCrest Training College

July 16, 2026

Abstract

This session bridges the gap between raw clinical data and robust machine learning models. We transition from exploratory analysis to building leak-proof preprocessing pipelines, training diverse classification algorithms, and establishing a rigorous baseline evaluation.

- **Pipelines:** Robust numerical and categorical preprocessing to prevent data leakage.
- **Model Selection:** Training 7 diverse classification algorithms to capture various data patterns.
- **Tuning:** `RandomizedSearchCV` with stratified cross-validation for efficient optimization.
- **Evaluation:** Comprehensive metrics on an untouched hold-out test set.

Our ultimate goal is to establish a reliable, high-performing baseline for predicting Atherosclerotic Heart Disease (AHD).

Session Objectives

- Build robust, leak-proof preprocessing pipelines using the `ColumnTransformer` concept.
- Handle numerical and categorical data appropriately to ensure data integrity.
- Understand and train multiple classification algorithms (LR, KNN, SVM, NB, DT, RF, XGBoost).
- Master the concepts behind hyperparameter tuning and cross-validation.
- Evaluate baseline models on an untouched hold-out test set using a comprehensive suite of clinical metrics.

Agenda

1. **Building Robust Preprocessing Pipelines** (Numerical, Categorical, ColumnTransformer)
2. **Model Selection & Hyperparameter Tuning Setup** (Algorithms, Grids, RandomizedSearchCV)
3. **Model Training Execution** (Running pipelines, extracting parameters)
4. **Baseline Evaluation on Hold-Out Test Set** (Metrics, Confusion Matrices, Comparison)

Part 1: Building Robust Preprocessing Pipelines

"Ensuring data integrity and preventing leakage."

The Role of Preprocessing in Machine Learning

The Reality of Clinical Data

Raw healthcare data is inherently messy, incomplete, and heterogeneous. It contains mixed data types, missing values, and varying scales.

- **Algorithmic Requirements:** Most ML algorithms require strictly numerical, clean, and scaled inputs to function correctly.
- **The Bridge:** Preprocessing transforms raw, real-world data into a mathematical representation that algorithms can interpret.
- **Impact:** The quality of preprocessing directly dictates the upper bound of model performance. "Garbage in, garbage out."

The Threat of Data Leakage in Preprocessing

Definition

Data leakage occurs when information from outside the training set (e.g., the test set) unintentionally influences the model training process.

- **The Preprocessing Trap:** Calculating global means, medians, or scaling factors using the *entire* dataset before splitting.
- **The Consequence:** The model indirectly "sees" the test data, leading to unrealistically high performance metrics.
- **The Reality:** When deployed, the model will fail catastrophically because it cannot access future data to calculate those global statistics.

Numerical Pipeline: Imputation Strategy

- **The Problem:** Missing numerical values (e.g., missing cholesterol readings) will cause most algorithms to crash.
- **The Strategy:** `Median Imputation`.

Why Median over Mean?

- Clinical data is often skewed and contains extreme outliers (e.g., exceptionally high blood pressure).
- The mean is highly sensitive to outliers and can distort the central tendency.
- The median is robust, providing a more representative "typical" value for the majority of patients.

Crucial Rule: The median is calculated *only* on the training folds and applied to the validation fold.

Numerical Pipeline: Feature Scaling

- **The Problem:** Features exist on vastly different scales (e.g., Age: 20-90 vs. Cholesterol: 100-400).
- **The Strategy:** `Standardization` (Z-score normalization).

The Mechanism

Transforms each feature by subtracting its mean and dividing by its standard deviation.

$$z = \frac{(x - \mu)}{\sigma}$$

Results in a distribution centered at 0 with a standard deviation of 1.

Why it Matters

Prevents features with larger magnitudes from disproportionately dominating the objective function in distance-based and gradient-based algorithms.

Categorical Pipeline: Imputation Strategy

- **The Problem:** Missing categorical values (e.g., unknown chest pain type).
- **The Strategy:** `Most Frequent` (Mode) Imputation.

Conceptual Approach

- Identifies the most common category within a specific feature based *strictly* on the training data.
- Replaces missing entries with this dominant category.
- Preserves the overall distribution and marginal probabilities of the nominal variables.

Note: For variables where "missingness" itself is clinically informative, creating a distinct "Missing" category might be preferred.

Categorical Pipeline: Handling Unknown Categories

The Real-World Challenge

When a model is deployed, it will inevitably encounter categorical values it never saw during training (e.g., a new hospital code).

- **The Failure Mode:** Standard encoders will crash or throw an error when faced with an unseen category.
- **The Solution:** Configure the encoder to handle unknowns gracefully.
- **The Mechanism:** Assign a specific, predefined encoded value (e.g., -1) to any unseen category.

Clinical Benefit

Allows the model to treat "unknown" as a distinct state, preventing system crashes and maintaining robustness in live environments.

The ColumnTransformer: Unifying the Workflow

The Orchestration Challenge

A single dataset contains both numerical and categorical columns, each requiring a completely different preprocessing sequence.

- **The Solution:** The `ColumnTransformer` acts as a router.
- **How it Works:** It applies the numerical pipeline to specified numerical columns and the categorical pipeline to specified categorical columns simultaneously.
- **The Output:** It seamlessly concatenates the transformed outputs into a single, unified numerical matrix.

This ensures that the distinct logic for each data type remains isolated, clean, and leak-proof.

Pipeline Architecture: Best Practices

1. **Encapsulation:** Bundle all preprocessing and modeling steps into a single, cohesive object.
2. **Leak-Proofing:** Ensure that every transformation that "learns" from data (imputation, scaling, encoding) is strictly confined to the training phase.
3. **Reproducibility:** A unified pipeline guarantees that the exact same transformations are applied during training, cross-validation, and final deployment.
4. **Deployment Readiness:** The final pipeline can be saved and loaded as a single artifact, eliminating the risk of preprocessing mismatches in production.

Part 2: Model Selection & Hyperparameter Tuning Setup

"Choosing the right algorithm and optimizing its configuration."

Model Selection: The "No Free Lunch" Theorem

The Core Concept

The "No Free Lunch" theorem states that no single machine learning algorithm universally outperforms all others across every possible dataset.

- **Algorithmic Diversity:** Different algorithms make different assumptions about the data (e.g., linearity, feature independence, hierarchical splits).
- **The Strategy:** Train a diverse ensemble of algorithms to capture various underlying patterns.
- **The Goal:** Let the data dictate the best approach by establishing a rigorous, empirical baseline across multiple paradigms.

Linear & Distance-Based Algorithms

- **Logistic Regression:**
  - Models the probability of a binary outcome using a linear combination of features.
  - *Strength:* Highly interpretable, fast, excellent baseline for linear boundaries.
- **Support Vector Machines (SVM):**
  - Finds the optimal hyperplane that maximizes the margin between classes.
  - *Strength:* Effective in high-dimensional spaces; kernels allow for complex, non-linear boundaries.
- **K-Nearest Neighbors (KNN):**
  - Classifies based on the majority class of the 'k' closest training instances.
  - *Strength:* Non-parametric, makes no assumptions about data distribution.

Probabilistic & Rule-Based Algorithms

- **Naive Bayes:**
  - Probabilistic classifier based on Bayes' theorem with a "naive" assumption of feature independence.
  - *Strength:* Extremely fast, performs surprisingly well in high-dimensional spaces despite the independence assumption.
- **Decision Trees:**
  - Splits data hierarchically based on feature thresholds that maximize information gain or minimize impurity.
  - *Strength:* Highly interpretable (white-box model), requires minimal data preprocessing, captures non-linear relationships naturally.

Ensemble Methods: The Power of Crowds

- **Random Forest (Bagging):**
  - Builds multiple deep decision trees on random subsets of data and features, then averages their predictions.
  - *Strength:* Significantly reduces variance and overfitting compared to a single tree.
- **XGBoost (Boosting):**
  - Builds trees sequentially, where each new tree attempts to correct the residual errors of the previous trees.
  - *Strength:* Reduces bias, highly optimized for speed and performance, consistently dominates tabular data competitions.

Introduction to Hyperparameter Tuning

Parameters vs. Hyperparameters

- **Parameters:** Internal configuration *learned* from the data during training (e.g., weights in Logistic Regression, split thresholds in Trees).
- **Hyperparameters:** External configuration *set by the practitioner* before training begins (e.g., learning rate, tree depth, number of neighbors).

- **The Goal of Tuning:** Find the optimal hyperparameter combination that maximizes the model's ability to generalize to unseen data.
- **The Challenge:** Balancing model complexity (underfitting vs. overfitting) without exhausting computational resources.

Defining the Search Space (Conceptual Grids)

Conceptualizing the Grid

A hyperparameter grid defines the specific values or distributions of hyperparameters that the tuning algorithm will explore.

- **Logistic Regression:** Regularization strength (`C`) and optimization algorithm (`solver`).
- **KNN:** Number of neighbors (`k`) and distance weighting scheme.
- **XGBoost:** Number of trees (`n_estimators`), maximum tree depth (`max_depth`), and step size shrinkage (`learning_rate`).

Strategy: Include a mix of broad ranges and fine-grained values to allow the search algorithm to navigate the space effectively.

Cross-Validation: Stratified 5-Fold

K-Fold Cross-Validation

The training data is divided into K equal subsets. The model is trained on $K-1$ folds and validated on the remaining fold. This process repeats K times.

- **Why 5-Fold?** Provides a robust, low-variance estimate of model performance while maintaining a reasonable computational cost.
- **The "Stratified" Imperative:** In standard K-Fold, random splits might result in folds with no positive cases (sick patients).
- **Stratification:** Ensures that every fold maintains the exact same proportion of classes as the original dataset, which is critical for reliable evaluation in imbalanced clinical data.

RandomizedSearchCV: Efficiency in Tuning

Grid Search vs. Random Search

- **Grid Search:** Exhaustively tries every single combination in the grid. Computationally explosive as dimensions increase.
- **Random Search:** Samples a fixed number of random combinations from the specified parameter distributions.

- **The Advantage:** Often finds a near-optimal combination much faster than Grid Search.
- **The Concept:** Not all hyperparameters are equally important. Random search allocates more trials to the most influential parameters by chance, making it highly efficient for complex models like XGBoost.

Choosing the Right Optimization Metric

The Problem with Accuracy

In imbalanced datasets (e.g., 90% healthy, 10% diseased), a model that always predicts "Healthy" achieves 90% accuracy but is clinically useless.

- **The Solution:** Optimize the tuning process using **ROC-AUC** (Receiver Operating Characteristic - Area Under Curve).
- **Why ROC-AUC?** It is threshold-independent. It evaluates the model's overall ability to rank a randomly chosen positive instance higher than a randomly chosen negative instance.
- **Benefit:** Provides a single, reliable scalar value to compare models during the tuning process, regardless of the class imbalance.

Part 3: Model Training Execution

"Running the pipelines and extracting insights."

Executing the Training Workflow

The Conceptual Loop

The training execution is an automated, systematic process applied to every candidate algorithm.

1. **Wrap:** Embed the chosen algorithm inside the unified preprocessing pipeline.
2. **Initialize:** Configure the search object with the specific hyperparameter grid and stratified cross-validation.
3. **Fit:** Execute the search on the training data. The system automatically handles the folding, preprocessing, training, and scoring.
4. **Parallelize:** Utilize all available computational cores to evaluate multiple hyperparameter combinations simultaneously.

Extracting the Best Estimator

Identifying the Champion

Once the search completes, the system identifies the hyperparameter combination that yielded the highest mean cross-validation ROC-AUC.

- **The "Best Estimator":** This is not just a set of parameters; it is a *fully fitted pipeline* trained on the entire training dataset using the optimal configuration.
- **Ready for Deployment:** Because it is a unified pipeline, this best estimator can immediately be used to predict on new, unseen data without any manual preprocessing steps.
- **Insight Extraction:** We can inspect the specific hyperparameters chosen to understand the optimal model complexity for our specific clinical dataset.

Interpreting Cross-Validation Results

Beyond the Mean Score

The average cross-validation score tells us the expected performance, but the **standard deviation** tells us about model stability.

- **Low Variance:** The model performs consistently well across different subsets of data. It is robust and reliable.
- **High Variance:** The model's performance fluctuates wildly depending on the training fold. This indicates high sensitivity to specific data points (overfitting) or an unstable algorithm.

Clinical Implication

In healthcare, a stable model (low variance) is often preferred over a slightly more accurate but highly volatile model, as consistency builds clinical trust.

The Trade-off: Performance vs. Training Time

Tracking Fit Time

During the search, we also track the time it takes to train each hyperparameter combination.

- **The Reality Check:** A complex ensemble model (like XGBoost) might yield a 2% higher ROC-AUC than Logistic Regression but take 100 times longer to train and predict.
- **Deployment Constraints:** If the model needs to run in real-time at the bedside, inference speed is critical.
- **The Decision:** We must balance the marginal gain in predictive performance against the computational cost and latency requirements of the clinical environment.

Part 4: Baseline Evaluation on Hold-Out Test Set

"The ultimate test of generalization."

The Hold-Out Test Set: Final Evaluation

The Concept of the "Untouched" Set

The test set has been completely isolated. It was not used for training, nor was it used for hyperparameter tuning.

- **Simulating the Future:** Evaluating on the test set simulates the model's performance on entirely new, future patients.
- **The Final Verdict:** Cross-validation scores are estimates; the test set score is the definitive, unbiased measure of the model's real-world generalization capability.
- **One-Shot Deal:** You should only evaluate on the test set once you have finalized your model selection. Repeatedly tuning based on test set results turns it into a training set.

Standard Classification Metrics

- **Accuracy:** The overall proportion of correct predictions. (Can be misleading in imbalanced datasets).
- **Precision (Positive Predictive Value):** Out of all patients the model *flagged* as diseased, how many actually had the disease?
  - *Focus:* Minimizing False Positives (unnecessary anxiety and tests).
- **Recall (Sensitivity):** Out of all patients who *actually* have the disease, how many did the model correctly flag?
  - *Focus:* Minimizing False Negatives (missed diagnoses).
- **F1-Score:** The harmonic mean of Precision and Recall. Provides a single metric that balances both concerns.

The Clinical Context: Precision vs. Recall

The Asymmetric Cost of Errors

In medical diagnostics, False Negatives and False Positives carry vastly different consequences.

- **Prioritizing Recall:** Missing a patient with Atherosclerotic Heart Disease (False Negative) could lead to a fatal cardiac event. We must catch as many true cases as possible.
- **Accepting Lower Precision:** Flagging a healthy patient for an extra echocardiogram (False Positive) causes financial cost and temporary anxiety, but is generally safe.

The Golden Rule

In this clinical context, we are willing to sacrifice Precision to maximize Recall. We would rather over-investigate than under-diagnose.

Threshold-Independent Metrics: ROC-AUC & PR-AUC

ROC-AUC (Receiver Operating Characteristic)

- Evaluates the model's ability to distinguish between classes across *all possible classification thresholds*.
- An AUC of 0.5 is random guessing; 1.0 is perfect separation.

PR-AUC (Precision-Recall Curve)

- Plots Precision against Recall across all thresholds.
- Highly sensitive to the performance on the *minority class* (the diseased patients).

Why use both? ROC-AUC gives a holistic view of separability, while PR-AUC provides a stricter, more realistic view of performance when the positive class is rare.

Visualizing Errors: Confusion Matrices & Comparison

The Confusion Matrix

A 2×2 table that visually breaks down predictions into True Positives, True Negatives, False Positives, and False Negatives.

- **Visual Diagnosis:** Instantly reveals if a model is biased (e.g., predicting the majority class almost exclusively).
- **Side-by-Side Comparison:** By generating confusion matrices for all tuned models, we can visually and quantitatively compare their error profiles.

Identifying the Champion

The initial top-performing model is not necessarily the one with the highest accuracy, but the one that optimally aligns its confusion matrix with our clinical goal: maximizing True Positives while minimizing False Negatives.

Summary & Best Practices

1. **Leak-Proof Pipelines:** Always encapsulate preprocessing within pipelines to ensure transformations are learned strictly from training data.
2. **Algorithmic Diversity:** Train multiple models to leverage the "No Free Lunch" theorem and find the best fit for your specific data.
3. **Rigorous Tuning:** Use Stratified Cross-Validation and Randomized Search to efficiently find optimal hyperparameters without overfitting.
4. **Clinical Metrics:** Evaluate models based on their real-world impact. Prioritize Recall and ROC-AUC over raw Accuracy in diagnostic settings.

Questions?

Lumumba W. Victor
Trainer — Data Analyst — ML Expert
MediaCrest Training College